Privilege Flow Graphs for Detecting Access Control Vulnerabilities via Graph Neural Networks

1. Problem Statement

1.1. The Nature of Access Control Vulnerabilities

Access control vulnerabilities occur when an application fails to properly enforce restrictions on what authenticated users are allowed to do. Unlike other vulnerability classes, access control is not a single coding mistake but a systemic property of the application's architecture. The enforcement logic is distributed across middleware, decorators, service functions, and database queries. A single missing check at any point in this chain can expose sensitive data or operations to unauthorized users.

The OWASP Top 10 (2021 edition) ranks Broken Access Control as the number one most critical web application security risk, up from fifth place in 2017. This ranking is based on incidence data from over 400,000 applications. CWE-862 (Missing Authorization) and CWE-639 (Authorization Bypass Through User-Controlled Key, which covers IDOR) are among the most frequently reported weaknesses in the National Vulnerability Database.

The consequences of access control failures are severe: unauthorized access to other users' data (privacy violations, regulatory penalties under GDPR/HIPAA), privilege escalation to administrative functions, and data modification or deletion by unauthorized parties.

1.2. Why Existing Approaches Fail

All existing automated vulnerability detection approaches, whether rule-based static analysis, taint-based analysis, or machine-learning-based methods, share a common architectural assumption: vulnerabilities are characterized by the presence of a detectable bad pattern. In taint analysis, this is a flow from a tainted source to a dangerous sink. In pattern-matching tools like Semgrep, it is a syntactic code pattern known to be insecure. In GNN-based detection (Devign, ReVeal, IVDetect, and their successors), it is a learned representation of vulnerable code at the function level.

Access control vulnerabilities violate this assumption fundamentally. The code that processes user data and queries the database is typically correct in isolation. There is no dangerous sink, no tainted flow to a risky function. The vulnerability is the absence of a check that should have been there: a missing @login_required decorator, a missing role verification, a missing ownership predicate in a database query. Detecting absence requires a model of what should be present, which in turn requires reasoning about the application's intended authorization structure, something no existing tool does.

Furthermore, existing GNN-based approaches operate at the function level. They analyze individual functions or code snippets. Access control is an application-level, cross-cutting concern. Whether an endpoint is properly protected depends on the entire call chain from HTTP request to database operation, spanning multiple functions, modules, and framework-provided middleware. No function-level representation can capture this.

1.3. The IDOR Sub-Problem

Insecure Direct Object Reference (IDOR) is a specific, highly prevalent, and highly damaging sub-class of access control vulnerabilities. It occurs when an application uses a user-supplied identifier (such as an ID in a URL path, query parameter, or request body) to directly look up a resource in the database without verifying that the requesting user is authorized to access that specific resource.

Example: A banking application exposes the endpoint GET /api/account/{id}/transactions. User Alice is authenticated and requests /api/account/1001/transactions (her own account). If Alice changes the URL to /api/account/1002/transactions and the application returns Bob's transactions without checking that Alice owns account 1002, this is an IDOR vulnerability.

The fix is straightforward: the database query should include a predicate that constrains results to the authenticated user (e.g., WHERE account.owner_id = current_user.id). The detection problem is identifying endpoints where this constraint is missing.

Despite IDOR being one of the most commonly reported vulnerability types in bug bounty programs (HackerOne reports it as the single most reported vulnerability class), there is no learned detection system for IDOR in any form in the literature. This represents a significant gap that this work directly addresses.

2. Proposed Abstraction: The Privilege Flow Graph (PFG)

2.1. Definition and Motivation

We introduce a new program representation called the Privilege Flow Graph (PFG). The PFG is a directed, attributed graph that encodes the authorization structure of an entire web application. It is purpose-built for reasoning about access control properties.

The key insight behind the PFG is that access control analysis does not require the full complexity of a Code Property Graph (which encodes every statement, every data dependency, every control flow edge). Instead, it requires a much smaller, higher-level graph that captures three things:

1. what are the entry points and what authorization do they require,
2. what are the sensitive operations and what privilege level do they demand,
3. what authorization checks exist on the paths between entry points and sensitive operations.

By abstracting away irrelevant code details (arithmetic, string manipulation, rendering logic, etc.), the PFG reduces an entire application from potentially millions of AST nodes to typically 100-500 PFG nodes. This makes GNN-based analysis tractable and focused.

2.2. Node Types and Annotations

The PFG contains four types of nodes, each with specific annotation attributes:

Endpoint Nodes: Represent HTTP-accessible entry points to the application.

- HTTP method (GET, POST, PUT, DELETE)
- Route pattern (e.g., /api/users/{id}/profile)
- Route parameters extracted (e.g., {id})
- Authentication required: boolean, derived from middleware/decorator analysis
- Required role: enumerated (NONE, USER, MODERATOR, ADMIN, SUPERADMIN)
- Ownership check present: boolean, indicates whether the endpoint verifies resource ownership

Middleware/Guard Nodes: Represent authorization enforcement points.

- Guard type: AUTHENTICATION (verifies identity), ROLE_CHECK (verifies role), OWNERSHIP_CHECK (verifies resource ownership)
- Condition: the specific check performed (e.g., 'role == ADMIN', 'resource.owner == request.user')
- Position: BEFORE_HANDLER or INLINE (whether it runs before the handler or within it)

Service/Function Nodes: Represent internal functions that are part of request processing.

- Inherited privilege level: the minimum privilege level of all callers
- Performs data access: boolean
- Performs mutation: boolean (write/update/delete operations)

Data Access Nodes: Represent database queries or external data operations.

- Operation type: READ, WRITE, UPDATE, DELETE
- Target entity/table (e.g., Invoice, User, Account)
- Has ownership predicate: boolean, whether the query includes a filter constraining results to the authenticated user
- Ownership predicate type: DIRECT (model.user == current_user), INDIRECT (join through related table), NONE
- User-controlled parameters used in query: list of parameter names

2.3. Edge Types

The PFG contains two types of directed edges:

Call Edges: Represent invocation relationships. An edge from node A to node B means that the code element represented by A calls/invokes the code element represented by B. These edges capture the structural control flow of request processing.

Example: Endpoint /api/invoices/{id} -> InvoiceService.get_invoice() -> InvoiceRepository.find_by_id().

Identifier Flow Edges: Represent the propagation of user-supplied identifiers through the call chain. An edge from node A to node B with label 'param:id' means that a user-controlled parameter named 'id' flows from A to B (possibly through intermediate transformations). These edges are critical for IDOR detection: they track whether a user-supplied identifier reaches a database query, and whether an ownership check intercepts it along the way.

2.4. PFG vs. Existing Program Representations

Representation	Granularity	Typical Size	Authorization Info	Access Control Suitability
Abstract Syntax Tree (AST)	Statement-level	Millions of nodes	None	Not suitable
Control Flow Graph (CFG)	Statement-level	Thousands per function	None	Not suitable
Code Property Graph (CPG)	Statement-level	Millions of nodes	None (must be inferred)	Theoretically possible, practically intractable
Call Graph	Function-level	Hundreds to thousands	None	Captures structure but lacks authorization semantics
Privilege Flow Graph (PFG)	Application-level	100-500 nodes	Rich annotations per node	Purpose-built for access control analysis

3. Detection Models

Two complementary GNN-based models operate on the PFG. They detect different failure modes and are designed to be used together. Their outputs are merged into a unified vulnerability report.

3.1. Model A: Authorization Anomaly Detector

Task: Node-level binary classification (each endpoint node is classified as NORMAL or ANOMALOUS).

Intuition: Within a well-built application, most endpoints follow consistent authorization patterns. Endpoints that access the same entity type (e.g., invoices) typically share the same authorization requirements. If 48 out of 50 endpoints that query the invoices table require authentication and include an ownership predicate, the 2 that do not are likely vulnerable.

Architecture:

- Input: The full PFG of the application, with all node and edge annotations encoded as feature vectors.
- GNN backbone: A Graph Attention Network (GAT) with 3-4 layers. Attention allows the model to weigh contributions from neighboring nodes differently, which is important because not all call chain relationships are equally relevant to authorization.
- Message passing: Each node aggregates authorization information from its neighbors. After K rounds of message passing, each endpoint node's embedding captures the authorization context of its entire call chain.
- Classification head: A two-layer MLP that takes the endpoint node embedding and outputs an anomaly score between 0 and 1.
- Training: Semi-supervised. The majority of endpoints in a well-maintained application are correctly protected and serve as implicit negative examples (not anomalous). A small number of known vulnerable endpoints (from synthetic injection or historical CVEs) serve as positive examples.

Handling False Positives:

- Intentionally public endpoints (health checks, login pages, public APIs) will appear anomalous to this model.
- Mitigation 1: The model is trained to recognize common public endpoint patterns (route patterns like /health, /login, /public/*).
- Mitigation 2: Developers can annotate endpoints with @public or equivalent markers that the PFG construction pipeline recognizes.
- Mitigation 3: A confidence threshold is applied, and borderline cases are surfaced as warnings rather than definitive findings.

3.2. Model B: Role-Boundary Path Analyzer

Task: Path-level binary classification (each path from an entry point to a sensitive operation is classified as PROPERLY_GUARDED or INSUFFICIENTLY_GUARDED).

Intuition: In a properly secured application, no path from a low-privilege entry point should reach a high-privilege operation without passing through an authorization gate that elevates the required privilege level. For example, a public (unauthenticated) endpoint should never be able to reach an administrative database operation (e.g., deleting all users) without passing through both an authentication check and an admin role verification.

Architecture:

- Input: The PFG with privilege level annotations on all nodes.
- Step 1 - Privilege Annotation: Each entry point is assigned a privilege level based on its annotations (NONE for public, USER for authenticated, ADMIN for admin-restricted). Each sensitive operation is assigned a required privilege level based on its nature (READ_OWN = USER, READ_ANY = ADMIN, WRITE = USER+OWNERSHIP, DELETE_ANY = ADMIN).
- Step 2 - Path Extraction: All paths from entry points to data access nodes are enumerated (with a depth limit to avoid path explosion, typically 6-8 hops, which covers the vast majority of real web application call chains).
- Step 3 - GNN Classification: Each path is encoded as a subgraph and passed through a GNN. The model classifies whether the authorization checks along the path are sufficient for the operation at the end of the path.
- Output: For each insufficiently guarded path, the model reports: the entry point, the sensitive operation, the required privilege level, the actual privilege level enforced, and the specific missing check.

3.3. IDOR Detection as a Special Case

IDOR detection is formalized as a specific instance of Model B's path analysis. The detection rule is:

A potential IDOR exists when ALL of the following conditions are true on a path from endpoint E to data access node D:

- E has a route parameter (e.g., {id}, {account_id}) that acts as a resource identifier.
- An identifier flow edge connects E to D, meaning the route parameter is used in the database query.
- D performs a data access operation (read, update, or delete) on an entity that has an ownership relationship to users.
- No node on the path from E to D performs an ownership check that constrains D's query to the authenticated user's resources.

This formalization is precise, testable, and captures the essential structure of IDOR without relying on heuristics or pattern matching. It can detect IDOR regardless of the specific framework, ORM, or coding style used, as long as the PFG is correctly constructed.

4. Scope and Target Frameworks

The PFG is a framework-independent abstraction, but constructing it requires framework-specific static analysis to extract authorization semantics from source code. We scope the initial implementation to two widely used web frameworks that represent two different authorization paradigms.

4.1. Django (Python)

Django uses a decorator-based authorization model. Key elements to extract:

- Endpoint discovery: URL patterns in urls.py, class-based views (ViewSet, APIView), function-based views.
- Authentication detection: @login_required decorator, LoginRequiredMixin, IsAuthenticated permission class (Django REST Framework).
- Role detection: @permission_required('app.permission_name'), custom permission classes, group-based checks.
- Ownership detection: ORM query patterns like Model.objects.filter(user=request.user), get_object_or_404() followed by ownership assertion.
- Data access: Django ORM calls (.filter(), .get(), .create(), .update(), .delete()), raw SQL queries.

4.2. Spring Boot (Java)

Spring Boot uses annotation-based and filter-chain-based authorization. Key elements to extract:

- Endpoint discovery: @RequestMapping, @GetMapping, @PostMapping, etc., on @Controller/@RestController classes.
- Authentication detection: Spring Security filter chain configuration (HttpSecurity.authorizeRequests()), @Secured annotation.
- Role detection: @PreAuthorize('hasRole(ADMIN)'), @RolesAllowed, .hasAuthority() in security config.
- Ownership detection: Repository query methods with principal parameter (e.g., findByIdAndUserId(id, userId)), @PreAuthorize with custom SpEL expressions checking ownership.
- Data access: Spring Data JPA repository calls, @Query annotations, EntityManager usage.

4.3. Extensibility to Other Frameworks

The PFG construction pipeline is designed with a pluggable extractor architecture. Each framework requires an extractor module that implements a common interface: discover endpoints, extract authorization annotations, build call edges, and trace identifier flow. Adding support for a new framework (e.g., Express.js/Node, ASP.NET, Laravel) requires writing a new extractor module without modifying the GNN models or the PFG representation itself.

5. Evaluation Strategy

5.1. Synthetic Benchmarks

We construct a benchmark suite of Django and Spring Boot applications with controlled access control properties. Each application contains:

- A set of correctly protected endpoints (positive examples of proper access control).
- A set of deliberately vulnerable endpoints with specific flaw types: missing authentication, missing role check, missing ownership predicate (IDOR), and combinations thereof.
- A set of intentionally public endpoints (to test false positive rates).
- Varying ratios of vulnerable to safe endpoints (1%, 5%, 10%, 20%) to test detection sensitivity.

The synthetic benchmark allows controlled experimentation: we know exactly which endpoints are vulnerable and what the vulnerability is, enabling precise measurement of detection accuracy.

5.2. Real-World Validation

We validate on real-world applications using two sources:

- Open-source applications with documented CVEs tagged CWE-862 (Missing Authorization) or CWE-639 (Authorization Bypass Through User-Controlled Key). We reconstruct the PFG from the vulnerable version and test whether the GNN correctly flags the vulnerable endpoint. We then verify the fix by constructing the PFG from the patched version.
- Open-source Django and Spring Boot applications from GitHub with significant user bases. We run the full pipeline and manually validate all findings (true positives, false positives, and false negatives discovered through manual review).

5.3. Metrics

Metric	Definition	Target
Precision	True positives / (True positives + False positives)	> 70% (actionable without excessive noise)
Recall	True positives / (True positives + False negatives)	> 80% (most real vulnerabilities caught)
F1 Score	Harmonic mean of Precision and Recall	> 75%
False Positive Rate	False positives / Total non-vulnerable endpoints	< 10%
IDOR-specific Recall	IDOR true positives / Total IDOR vulnerabilities	> 85%

5.4. Baselines

We compare against:

- Semgrep with community access-control rules: represents the state of the art in pattern-based detection.
- CodeQL with authorization-related queries (CWE-862, CWE-639): represents the state of the art in semantic static analysis.

- Manual code review by security-experienced developers (timed): provides a human baseline for comparison of effectiveness and efficiency.
- If available, any commercial SAST tool results on the same benchmark applications.

6. Differentiation from Prior Work

Approach	Representation	Granularity	Detects Absence?	Access Control?
Checkmarx / Fortify (Taint)	Data flow graph	Function/file	No	No (no concept of missing checks)
Semgrep (Pattern)	AST patterns	Statement	No	Limited (brittle rules)
CodeQL (Semantic)	Relational DB of code	Statement/function	Partially	Limited (manual query writing)
Devign (GNN)	CPG	Function	No	No
ReVeal (GNN)	CPG	Function	No	No
IVDetect (GNN)	PDG subgraph	Function	No	No
LineVul (Transformer)	Token sequence	Function/line	No	No
This work (PFG + GNN)	Privilege Flow Graph	Application	Yes	Yes (purpose-built)

The fundamental differentiator is that PFG + GNN is the only approach that (1) operates at the application level rather than the function level, (2) encodes authorization semantics directly in the graph representation, and (3) can detect the absence of required checks rather than only the presence of bad patterns.

7. Summary of Contributions

This work makes four distinct contributions:

Contribution 1: The Privilege Flow Graph (PFG)

- A novel program representation that encodes the authorization structure of a web application as a directed, attributed graph.
- Reduces application complexity from millions of AST nodes to hundreds of authorization-relevant nodes.
- Formally defined with four node types, two edge types, and rich annotation schemas.

Contribution 2: Two Complementary GNN-Based Detection Models

- Model A (Anomaly Detector): learns per-application authorization norms and flags deviations, addressing the intra-application consistency signal.
- Model B (Path Analyzer): detects privilege escalation paths and insufficiently guarded routes to sensitive operations, addressing the structural correctness signal.

Contribution 3: First Learned Detection System for IDOR

- Formalizes IDOR detection as the absence of an ownership predicate on an identifier flow path.
- No prior learned system for IDOR detection exists in the literature.

Contribution 4: Framework-Aware PFG Construction Pipeline

- Static analysis extractors for Django and Spring Boot that produce PFGs from source code without runtime instrumentation.
- Pluggable architecture allowing extension to additional frameworks.

8. Complete Implementation Roadmap: From Zero to Finish

This section provides a complete, sequential roadmap for implementing the PFG-based access control vulnerability detection system. Each phase includes its objectives, concrete tasks, deliverables, and dependencies on prior phases.

Phase 0: Foundation and Literature Study

Task 0.1: Deep Literature Review

- Study all GNN-based vulnerability detection papers from the SMS (Devign, ReVeal, IVDetect, LineVul, FUNDED, DeepWukong...) to understand their graph representations and why they cannot handle access control.
- Study access control vulnerability taxonomies: CWE-862 (Missing Authorization), CWE-863 (Incorrect Authorization), CWE-639 (IDOR), CWE-284 (Improper Access Control).
- Study existing static analysis approaches for access control: Semgrep rules, CodeQL queries, academic papers on authorization verification.
- Study GNN architectures relevant to the task: GAT (Graph Attention Networks), GraphSAGE, GIN (Graph Isomorphism Networks), and heterogeneous graph neural networks.
- Study anomaly detection on graphs: deep graph anomaly detection surveys, one-class classification approaches.

Task 0.2: Framework Authorization Internals Study

- Deep study of Django's authorization system: middleware, decorators, DRF permissions, ORM query patterns, url routing.
- Deep study of Spring Security: filter chains, annotation-based security, SpEL expressions, Spring Data JPA query derivation.
- Document every authorization pattern and idiom used in each framework. This document becomes the specification for the PFG extractors.

Task 0.3: Project Infrastructure Setup

- Set up version-controlled repository with clear directory structure.
- Set up Python environment with dependencies: PyTorch, PyTorch Geometric (PyG), NetworkX, tree-sitter (for parsing), javalang (for Java parsing).
- Set up experiment tracking (MLflow or Weights & Biases) for model training runs.
- Set up evaluation framework: scripts for computing precision, recall, F1, confusion matrices.

Deliverables:

- Literature review document with gap analysis confirming novelty.
- Framework authorization pattern catalog (Django + Spring Boot).
- Project repository with infrastructure ready.

Phase 1: PFG Formal Specification and Data Collection

Task 1.1: Formal PFG Specification

- Write the formal definition of the PFG as a mathematical object: G = (V, E, X_v, X_e) where V is the node set, E is the edge set, X_v are node features, X_e are edge features.
- Define the node feature vector schema: what features, what data types, what encoding (one-hot, ordinal, binary).
- Define the edge feature vector schema.
- Define the labeling scheme: what constitutes a 'vulnerable' endpoint, what constitutes a 'properly protected' endpoint.

- Write the formal IDOR detection predicate as a path property.

Task 1.2: Synthetic Application Construction

- Build 5-8 Django applications of varying complexity (10 endpoints to 80+ endpoints each).
- Build 5-8 Spring Boot applications of equivalent complexity.
- Each application must have: (a) A coherent domain model (e-commerce, healthcare, banking, social media, etc.). (b) A mix of properly protected and deliberately vulnerable endpoints. (c) All four vulnerability sub-types: missing authentication, missing role check, missing ownership/IDOR, and combinations. (d) Intentionally public endpoints that should NOT be flagged. (e) Documented ground truth: every endpoint labeled as SAFE, VULNERABLE (with type), or INTENTIONALLY_PUBLIC.
- Vary the vulnerable-to-safe ratio across applications to test sensitivity.

Task 1.3: Real-World Application Collection

- Search GitHub for open-source Django and Spring Boot applications with: (a) Meaningful access control (user roles, ownership concepts). (b) Documented CVEs for access control flaws (ideal) or significant enough codebase to contain likely flaws. (c) Sufficient size (20+ endpoints) to provide meaningful PFGs.
- Collect at least 5 Django and 5 Spring Boot real-world applications.
- For applications with CVEs: document the vulnerable endpoint, the CVE details, and the fix applied.
- For applications without CVEs: plan for manual security review to establish ground truth.

Task 1.4: Dataset Documentation

- Create a complete dataset description: number of applications, number of endpoints, vulnerability distribution, framework breakdown.
- This will become the 'Dataset' section of the paper.

Deliverables:

- Formal PFG specification document.
- Synthetic benchmark suite (10-16 applications) with ground truth labels.
- Real-world application corpus (10+ applications) with partial ground truth.

Phase 2: Static Analysis Pipeline (PFG Construction)

Task 2.1: Django PFG Extractor

- Sub-task 2.1.1 - Endpoint Discovery: Parse urls.py files recursively to extract all URL patterns. Resolve class-based views (ViewSet action methods, APIView methods) and function-based views. Map each URL pattern to its handler function. Output: list of (route_pattern, http_method, handler_function) tuples.
- Sub-task 2.1.2 - Authorization Annotation: For each endpoint, analyze the decorator chain (@login_required, @permission_required, @api_view with permission_classes). For class-based views, analyze the permission_classes attribute and authentication_classes attribute. For middleware-based auth, analyze Django MIDDLEWARE settings and track which middleware applies to which URL patterns. Output: each endpoint annotated with (requires_auth, required_role, has_ownership_check).
- Sub-task 2.1.3 - Call Graph Construction: Starting from each endpoint handler, trace function calls using AST analysis (Python ast module or tree-sitter). Resolve imports and method calls

within the project. Handle Django-specific patterns: Model.objects.filter(), serializer.save(), signal handlers. Limit depth to 8 hops (configurable). Output: directed call graph edges.
- Sub-task 2.1.4 - Data Access Detection: Identify all Django ORM calls: .filter(), .get(), .exclude(), .create(), .update(), .delete(), .aggregate(), raw(). For each ORM call, determine: target model, operation type, filter predicates. Check if filter predicates include the authenticated user (request.user appears in filter kwargs). Output: data access nodes with ownership annotations.
- Sub-task 2.1.5 - Identifier Flow Tracking: For each route parameter (extracted from URL pattern), trace its usage through the handler function and into service functions and ORM calls. Use intra-procedural data flow analysis (variable assignments and function arguments). Output: identifier flow edges from endpoint nodes to data access nodes, labeled with parameter name.
- Sub-task 2.1.6 - PFG Assembly: Combine all extracted information into a single PFG per application. Serialize as a PyG Data object (node features tensor, edge index tensor, edge features tensor, labels tensor).

Task 2.2: Spring Boot PFG Extractor

- Same sub-task structure as 2.1, adapted for Spring Boot/Java:
- 2.2.1 - Endpoint Discovery: Parse @Controller/@RestController classes. Extract @RequestMapping, @GetMapping, etc. Map to handler methods.
- 2.2.2 - Authorization Annotation: Parse Spring Security configuration (WebSecurityConfigurerAdapter or SecurityFilterChain). Extract @PreAuthorize, @Secured, @RolesAllowed annotations. Analyze HttpSecurity.authorizeRequests() chain (.antMatchers().hasRole(), .authenticated(), .permitAll()).
- 2.2.3 - Call Graph: Trace from controller methods through @Service classes to @Repository classes. Handle dependency injection (constructor injection, @Autowired). Use javalang or tree-sitter-java for parsing.
- 2.2.4 - Data Access: Identify Spring Data JPA repository method calls. Parse @Query annotations and derived query method names. Check for principal/user parameters in repository queries.
- 2.2.5 - Identifier Flow: Trace @PathVariable and @RequestParam usage through the call chain to repository method arguments.
- 2.2.6 - PFG Assembly: Same PyG serialization as Django.

Task 2.3: Extractor Validation

- Run both extractors on all synthetic benchmark applications.
- Manually verify that every PFG matches the expected graph structure (correct nodes, correct edges, correct annotations).
- Fix any extraction errors. This step is critical: if the PFG is wrong, the GNN cannot learn correctly.
- Compute extraction accuracy: what percentage of authorization annotations are correctly identified?

Task 2.4: PFG Visualization

- Build a visualization tool (using NetworkX + matplotlib or Graphviz) that renders PFGs with node type shapes, annotation labels, and edge type colors.
- This is useful for debugging and will also provide figures for the paper.

Deliverables:

- Django PFG extractor (Python package, fully tested).

- Spring Boot PFG extractor (Python package, fully tested).
- PFG dataset: all benchmark applications converted to PFG format (PyG Data objects).
- Extraction accuracy report.
- PFG visualization tool.

Phase 3: GNN Model Development

Task 3.1: Feature Engineering

- Convert all PFG node annotations to numerical feature vectors. Design the encoding scheme:
- Node type: one-hot encoding (4 values: endpoint, middleware, service, data_access).
- HTTP method: one-hot encoding (5 values: GET, POST, PUT, DELETE, PATCH).
- Requires authentication: binary (0 or 1).
- Required role: ordinal encoding (0=NONE, 1=USER, 2=MODERATOR, 3=ADMIN, 4=SUPERADMIN).
- Has ownership check: binary.
- Operation type: one-hot encoding (4 values: READ, WRITE, UPDATE, DELETE).
- Has ownership predicate: binary.
- Has route parameter: binary.
- Route parameter flows to query: binary.
- Edge features: edge type (call vs. identifier_flow), one-hot encoded.
- Normalize all features to [0, 1] range.

Task 3.2: Model A Implementation (Anomaly Detector)

- Implement GAT-based encoder: 3 GAT layers with multi-head attention (4 heads), hidden dimension 64, output dimension 32.
- Implement MLP classification head: Linear(32, 16) -> ReLU -> Dropout(0.3) -> Linear(16, 1) -> Sigmoid.
- Loss function: Binary cross-entropy with class weighting (vulnerable endpoints are rare, so upweight the positive class).
- Implement per-entity-type anomaly scoring: group endpoints by the entity type they access, compute anomaly relative to the group rather than the entire application.
- Alternative architecture to try: Graph autoencoder (reconstruct normal PFG patterns, flag high reconstruction error as anomalous).

Task 3.3: Model B Implementation (Path Analyzer)

- Path extraction: enumerate all paths from endpoint nodes to data access nodes in the PFG (DFS/BFS with depth limit).
- Path encoding: each path is a subgraph. Extract the induced subgraph for each path.
- Implement GNN for path classification: apply a GNN to the path subgraph, then apply graph-level readout (mean/max pooling of node embeddings) followed by MLP classifier.
- Alternative: encode each path as a sequence of node feature vectors and use an LSTM or Transformer to classify.
- Label each path: PROPERLY_GUARDED if the authorization gates on the path are sufficient for the operation at the path's terminal node, INSUFFICIENTLY_GUARDED otherwise.
- Loss function: Binary cross-entropy.
- Special IDOR head: a separate output that specifically predicts whether an IDOR exists (identifier flows to query with no ownership check).

Task 3.4: Training Pipeline

- Data splitting: split by application (not by endpoint) to avoid data leakage. Train on N-2 applications, validate on 1, test on 1. Use cross-validation across applications.
- Training loop: Adam optimizer, initial learning rate 0.001 with cosine annealing schedule.
- Early stopping: monitor validation F1, stop after 20 epochs without improvement.
- Hyperparameter search: grid search over learning rate (0.01, 0.001, 0.0001), hidden dimension (32, 64, 128), number of GAT layers (2, 3, 4), number of attention heads (2, 4, 8), dropout rate (0.1, 0.3, 0.5).
- Track all experiments in MLflow/W&B.

Task 3.5: Model Ablation Studies

- Ablate node features: remove authorization annotations one at a time to measure their contribution.
- Ablate edge types: remove identifier flow edges to measure their importance for IDOR detection.
- Ablate GNN depth: vary number of layers to find the sweet spot.
- Compare GNN architectures: GAT vs. GCN vs. GraphSAGE vs. GIN.
- Compare against non-GNN baselines: random forest on node features (no graph structure), rule-based heuristics on PFG (no learning).

Deliverables:

- Trained Model A and Model B with best hyperparameters.
- Training curves and ablation study results.
- Cross-validation results across all benchmark applications.

Phase 4: Evaluation and Benchmarking

Task 4.1: Synthetic Benchmark Evaluation

- Run the full pipeline (PFG construction + both models) on all synthetic benchmark applications.
- Compute per-application and aggregate metrics: precision, recall, F1, false positive rate.
- Break down results by vulnerability sub-type: missing auth, missing role, IDOR, combined.
- Analyze errors: categorize all false positives (legitimate exception? extraction error? model error?) and all false negatives (what did the model miss and why?).

Task 4.2: Real-World Evaluation

- Run the pipeline on all collected real-world applications.
- For CVE applications: check whether the known vulnerable endpoint is flagged (recall on known vulns).
- For non-CVE applications: manually review all findings. Attempt to confirm true positives by actually exploiting the vulnerability (in a local test environment). Document false positives with explanations.
- If any previously unknown vulnerabilities are discovered, consider responsible disclosure.

Task 4.3: Baseline Comparison

- Run Semgrep with all available access-control rules on the same applications. Compute the same metrics.
- Run CodeQL with CWE-862 and CWE-639 queries on the same applications (requires compilation for Java). Compute the same metrics.
- Conduct timed manual review: have 2-3 developers independently review a subset of applications for access control flaws. Measure time spent and findings.

- Produce a comparison table showing PFG+GNN vs. Semgrep vs. CodeQL vs. manual review on precision, recall, F1, and time.

Task 4.4: Scalability Analysis

- Measure PFG construction time as a function of application size (number of endpoints, lines of code).
- Measure GNN inference time as a function of PFG size.
- Demonstrate that the approach scales to production-sized applications (hundreds of endpoints).

Deliverables:

- Comprehensive evaluation report with all metrics, tables, and error analysis.
- Baseline comparison table.
- Scalability analysis results.

Phase 5: Integration

- Build a unified CLI tool that takes a project directory path and framework type as input and produces a vulnerability report as output.
- Report includes:

a. PFG visualization of the application.
b. List of flagged endpoints with vulnerability type, confidence score, and explanation.
c. For each finding: the path from entry point to sensitive operation, the missing check, and a remediation suggestion.

- Package as an installable Python tool (pip install).

Deliverables:

- Integrated CLI tool.
- Complete paper draft.